

A Practical Guide to BERT language model

TOPIC -- BERT LANGUAGE MODEL

-- 01 --

Architectural family The encoder stack of BERT has 2 free parameters: L , the number of layers, and H , the hidden size. There are always $H / 64$ self-attention heads, and the feed-forward/filter size is always $4H$. By varying these two numbers, one obtains an entire family of BERT models. For BERT:

-- 02 --

Cost BERT was trained on the BookCorpus (800M words) and a filtered version of English Wikipedia (2,500M words) without lists, tables, and headers. Training BERTBASE on 4 cloud TPU (16 TPU chips total) took 4 days, at an estimated cost of 500 USD. Training BERTLARGE on 16 cloud TPU (64 TPU chips total) took 4 days.

-- 03 --

Bidirectional encoder representations from transformers (BERT) is a language model introduced in October 2018 by researchers at Google. It learns to represent text as a sequence of vectors using self-supervised learning. It uses the encoder-only transformer architecture. BERT dramatically improved the state of the art for large language models. As of 2020, BERT is a ubiquitous baseline in natural language processing (NLP) experiments. BERT is trained by masked token prediction and next sentence prediction. With this training, BERT learns contextual, latent representations of tokens in their context, similar to ELMo and GPT-2. It found applications for many natural language processing tasks, such as coreference resolution and polysemy resolution. It improved on ELMo and spawned the study of "BERTology", which attempts to interpret what is learned by BERT. BERT was originally implemented in the English language at two model sizes, BERTBASE (110 million parameters) and BERTLARGE (340 million parameters). Both were trained on the Toronto BookCorpus (800M words) and English Wikipedia (2,500M words). The weights were released on GitHub. On March 11, 2020, 24 smaller models were released, the smallest being BERTTINY with just 4 million parameters.

-- 04 --

Variants The BERT models were influential and inspired many variants. RoBERTa (2019) was an engineering improvement. It preserves BERT's architecture (slightly larger, at 355M parameters), but improves its training, changing key hyperparameters, removing the next-sentence prediction task, and using much larger mini-batch sizes. XLM-RoBERTa (2019) was a multilingual RoBERTa model. It was one of the first works on multilingual language modeling at scale. DistilBERT (2019) distills BERTBASE to a model with just 60% of its parameters (66M), while preserving 95% of its benchmark scores. Similarly, TinyBERT (2019) is a distilled model with just 28% of its parameters. ALBERT (2019) used shared-parameter across layers, and experimented with independently varying the hidden size and the word-embedding layer's output size as two hyperparameters. They also replaced the next sentence prediction task with the sentence-order prediction (SOP) task, where the model must distinguish the correct order of two consecutive text segments from their reversed order. ELECTRA (2020) applied the idea of generative adversarial networks to the MLM task. Instead of masking out tokens, a small language model generates random plausible substitutions, and a larger network identify these replaced tokens. The small model aims to fool the large model. DeBERTa (2020) is a significant architectural variant, with disentangled attention. Its key idea is to treat the positional and token encodings separately throughout the attention mechanism. Instead of combining the positional encoding (x_{position}) and token encoding (x_{token}) into a single input vector ($x_{\text{input}} = x_{\text{position}} + x_{\text{token}}$), DeBERTa keeps them separate as a tuple: $(x_{\text{position}}, x_{\text{token}})$. Then, at each self-attention layer, DeBERTa computes three distinct attention matrices, rather than the single attention matrix used in BERT:

-- 05 --

Masked language modeling (MLM): In this task, BERT ingests a sequence of words, where one word may be randomly changed ("masked"), and BERT tries to predict the original words that had been changed. For example, in the sentence "The cat sat on the [MASK]," BERT would need to predict "mat." This helps BERT learn bidirectional context, meaning it understands the relationships between words not just from left to right or right to left but from both directions at the same time. Next sentence prediction (NSP): In this task, BERT is trained to predict whether one sentence logically follows another. For example, given two sentences, "The cat sat on the mat" and "It was a sunny day", BERT has to decide if the second sentence is a valid continuation of the first one. This helps BERT understand relationships between sentences, which is important for tasks

like question answering or document classification.

-- 06 --

Given two sentences, the model predicts if they appear sequentially in the training corpus, outputting either [IsNext] or [NotNext]. During training, the algorithm sometimes samples two sentences from a single continuous span in the training corpus, while at other times, it samples two sentences from two discontinuous spans. The first sentence starts with a special token, [CLS] (for "classify"). The two sentences are separated by another special token, [SEP] (for "separate"). After processing the two sentences, the final vector for the [CLS] token is passed to a linear layer for binary classification into [IsNext] and [NotNext]. For example:

-- 07 --

Embedding This section describes the embedding used by BERTBASE. The other one, BERTLARGE, is similar, just larger. The tokenizer of BERT is WordPiece, which is a sub-word strategy like byte-pair encoding. Its vocabulary size is 30,000, and any token not appearing in its vocabulary is replaced by [UNK] ("unknown").

-- 08 --

In masked language modeling, 15% of tokens would be randomly selected for masked-prediction task, and the training objective was to predict the masked token given its context. In more detail, the selected token is:

-- 09 --

BERT is meant as a general pretrained model for various applications in natural language processing. That is, after pre-training, BERT can be fine-tuned with fewer resources on smaller datasets to optimize its performance on specific tasks such as natural language inference and text classification, and sequence-to-sequence-based language generation tasks such as question answering and conversational response generation. The original BERT paper published results demonstrating that a small amount of finetuning (for BERTLARGE, 1 hour on 1 Cloud TPU) allowed it to achieved state-of-the-art performance on a number of natural language understanding tasks: